Análise do Impacto de Variações de Prompting e Feedback Semântico no ARC-AGI-1: Fundamentação Estatística

Fabio Luiz da Costa Cieslak[*1], **Izadora Candotti de Oliveira**[*1], **Matheus Machado**[*1]

Universidade Federal do Rio Grande do Sul[1]

Abstract

O benchmark *Abstraction and Reasoning Corpus* (ARC-AGI-1) avalia o raciocínio indutivo e a generalização em sistemas de inteligência artificial por meio da transformação de grades matriciais coloridas a partir de demonstrações visuais. Neste trabalho investiga-se a síntese indutiva de programas guiada por contraexemplos (CEGIS) e o efeito de restrições textuais Anti-Trapaça sobre oito modelos de linguagem em 100 tarefas do benchmark. A variante Anti-Trapaça, que restringe desvios condicionais ad-hoc e demanda a declaração prévia do invariante abstrato, revelou-se estatisticamente equivalente ao CEGIS padrão ($50{,}4\%$ vs. $49{,}8\%$, $p = 0{,}5515$), refutando a hipótese de que diretivas de prompt anti-hardcoding oferecem vantagem generalizável. A análise dos modos de falha revelou que, embora o teto de busca responda por $86{,}9\%$ dos erros no agregado, o sobreajuste espúrio é a barreira dominante na fronteira (100% no `Grok 4.6` e $71{,}4\%$ no `DeepSeek V4`).

1 Introdução

O *Abstraction and Reasoning Corpus* (ARC-AGI) (Chollet 2019) foi proposto para mensurar a inteligência geral por meio da indução, contrastando com a memorização em bases massivas característica dos Grandes Modelos de Linguagem (LLMs). Cada tarefa é composta por matrizes bidimensionais discretas com valores inteiros de 0 a 9 (representando dez cores distintas), acompanhadas de 2 a 5 pares de treino (entrada e saída esperada) e uma grade de teste oculta, cuja transformação deve ser inferida com exatidão.

Neste trabalho, explora-se a síntese indutiva de programas guiada por contraexemplos (CEGIS, *Counterexample-Guided Inductive Synthesis*) (Solar-Lezama 2008) para resolver tarefas ARC-AGI usando LLMs. A indução de programas a partir de demonstrações matriciais alinha-se a marcos do domínio, como o aprendizado de invariantes e linguagens conceituais no DreamCoder (Ellis et al. 2023) e a comunicação de programas naturais no corpus LARC (Acquaviva et al. 2022). Em contraste com métodos dedutivos clássicos baseados em provadores lógicos e especificações completas, o CEGIS sintetiza funções em Python sob supervisão de exemplos empíricos. O LLM atua como sintetizador probabilístico, enquanto o interpretador atua como verificador determinístico. Essa dinâmica conecta-se aos paradigmas de auto-reparo e reflexão verbal em LLMs, como *Self-Refine* (Madaan et al. 2023) e *Reflexion* (Shinn et al. 2023), nos quais a cadeia de raciocínio (Wei et al. 2022) é direcionada pela evidência do erro.

Devido à estrutura do CEGIS, é possível que ocorram soluções parciais para as tarefas, nas quais o modelo adiciona condicionais ou dados específicos das tarefas de treino (hardcoded) que não generalizam para as tarefas de teste. Para investigar esse fenômeno, propõe-se um teste entre dois prompts de contraexemplo: um com diretivas denominadas Anti-Trapaça e outro sem. O objetivo dessas regras é coibir memorizações parciais por meio do veto a regras ad-hoc e da exigência de verbalização do invariante subjacente. A hipótese central deste trabalho é que *restrições semânticas anti-hardcoding e a reflexão explícita sobre invariantes melhoram a taxa de acerto no teste oculto do ARC-AGI*. Para isso, estabelece-se um protocolo experimental pareado no benchmark, avaliando formalmente oito modelos de linguagem em uma amostra controlada de 100 tarefas.

2 Síntese de Programas e Variações de Prompting

2.1 Ciclo Conversacional do CEGIS Semântico

Diferentemente de avaliações baseadas em amostragem paralela (*pass@k* com múltiplas gerações independentes), neste trabalho o *pass@1* designa estritamente o Turno 0 de uma sessão conversacional interativa *stateful* com orçamento de reparos $K_{\max} = 5$. No Turno 0, o modelo recebe as demonstrações de treino e sintetiza a função candidata `transform(grid)`. Para isolar o código executável do texto explicativo, o sistema utiliza expressões regulares para extrair exclusivamente o bloco delimitado por ```` ```python ... ``` ````, prevenindo falhas de interpretação decorrentes de prosa adicional.

A dinâmica iterativa obedece a três políticas formais: (i) **Parada Estrita (*Hard Stopping*):** em cada turno $k \in \{0, \ldots, K_{\max} - 1\}$, a função candidata é executada sequencialmente sobre os pares de treino. Obtendo 100% de conformidade no treino, o ciclo de busca é encerrado imediatamente e o código é submetido à grade de teste oculta.

[*]These authors contributed equally.

Copyright © 2027, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Logo, soluções que acertam o treino no Turno 0 (*pass@1*) são preservadas sem alterações, impossibilitando regressão no treino; (ii) **Interrupção no Primeiro Contraexemplo:** havendo erro de sintaxe, *timeout* de 2,0s ou divergência matricial, a execução interrompe no primeiro exemplo discordante, gerando feedback corretivo; e (iii) **Esgotamento de Orçamento:** atingindo $K_{\max} = 5$ iterações sem convergência no treino, a sessão encerra-se como falha e o teste cego não é consultado.

2.2 Diretivas Anti-Trapaça e Elicitação de Invariantes

Modelos submetidos a feedback de contraexemplos podem cair em sobreajuste (*overfitting*), inserindo cláusulas condicionais dedicadas unicamente a satisfazer o exemplo discordante recém-apresentado. A estratégia Anti-Trapaça estabelece instruções negativas expressas contra ramificações ad-hoc e exige que o modelo explicite o conceito invariante antes de redigir o código.

Ressalta-se que a restrição à coordenadas fixas e a exigência do invariante foram aplicadas exclusivamente via engenharia de prompt conversacional, sem filtragem ou rejeição programática em tempo de execução no sandbox. Como ferramenta auditora *post-hoc*, utilizou-se uma varredura baseada em expressões regulares sobre as 2.398 funções geradas, inspecionando padrões de coordenadas e matrizes literais.

2.3 Especificação Estruturada dos Prompts

Para assegurar a estrita replicabilidade do estudo, o protocolo estabeleceu o texto literal dos blocos de instrução:

1. Prompt de Sistema: *"You are an expert AI solving ARC-AGI puzzles by writing Python code."*

2. Prompt Inicial de Passada Única (pass@1): Apresenta os pares de treino serializados em JSON (`Example 0`, etc.), requisita a função `transform(grid)` e estipula o bloco `ANTI_IMPORT`:

```
CODE RULES:
- DO NOT use any `import` statements
(e.g., collections, numpy, math, etc.).
- USE ONLY pure native Python types and
functions (list, dict, set, tuple,
range, len, min, max, sum, enumerate,
zip, abs).
```

3. Feedback de Contraexemplo (CEGIS): As grades 2D são serializadas em matrizes JSON (`json.dumps()`, e.g., `[[0,1],[2,3]]`). Sob falha no treino i, o sistema emite o feedback literal:

```
Your code failed on Training Example
{i}.
- Input: <json_input_grid>
- Expected Output: <json_expected_grid>
- Produced Output:
<json_produced_grid | Execution Error:
...>
[CODE RULES ...]
Analyze the discrepancy, fix your
logic, and provide the updated
`transform(grid)` function inside a
"`python ... "` block.
```

4. Diretivas Adicionais (Anti-Trapaça): Anexa textualmente ao feedback de contraexemplo o bloco literal `ANTI_CHEAT`:

```
- DO NOT add conditional branches
(if/else) targeting specific example
indices, hardcoded coordinates, or
specific row values of a single
training case. The transformation must
be a single, uniform
mathematical/geometric rule that
applies to ALL grids identically.
- Before writing the code, explicitly
state the single abstract invariant
that explains why the previous code
failed on Example X and how the new
logic naturally handles Examples 0, 1,
and 2 without special cases.
```

3 Metodologia e Fundamentação Estatística

3.1 Análise Pareada, Permutação e Bootstrap

Para comparar o desempenho das estratégias mantendo a estrutura pareada, utilizamos um teste de permutação com 100.000 réplicas. O teste exato de McNemar foi aplicado aos pares discordantes, e a reamostragem bootstrap pareada (10.000 réplicas) determinou os intervalos de confiança de 95%. O controle da taxa de erro familiar para os 8 modelos avaliados foi feito com a correção de Bonferroni ($\alpha = 0,00625$).

3.2 Classificação dos Modos de Falha

As falhas na síntese são categorizados em: (i) **Sobreajuste Espúrio (Falsa Convergência):** o código obtém sucesso em 100% das demonstrações de treino, porém falha na grade de teste cego devido à absorção de regras acidentais; e (ii) **Teto de Busca e Representação:** o processo não converge no conjunto de treino dentro das $K_{\max} = 5$ iterações concedidas.

4 Protocolo Experimental e Execução

4.1 Sandbox e Segurança de Execução

O código sintetizado é executado em subprocesso isolado via `multiprocessing` com inicialização estrita em contexto *spawn* e limite de tempo de 2,0s. Interceptores nativos do interpretador (`sys.addaudithook`) bloqueiam chamadas de disco (`open`), conectores de rede (`socket`), criação de processos (`subprocess`) e importações (`import`). A execução é restrita a uma lista de tipos e funções (`_SAFE_BUILTINS`). O critério de validação é exato e estrito: as matrizes de saída geradas devem coincidir com a referência em dimensão e coloração.

4.2 Modelos, Amostragem e Parâmetros de Inferência

Avaliou-se um painel com oito modelos: de fronteira (`Grok 4.6`, `DeepSeek V4 Flash 0731` e `GPT-5.6 Luna`), intermediários (`Claude Sonnet 5`, `Gemini 3.5 Flash Lite` e `Gemini 3.1 Flash Lite`) e compactos (`Leanstral 1.5.1` e `GPT-5.4 Nano`). A

Tabela 1: Resultados consolidados nos 100 problemas avaliados do ARC-AGI-1 sob os regimes de tentativa única (pass@1), CEGIS e CEGIS com Anti-Trapaça ($N = 100$). Acurácias expressas em percentual; as colunas de tarefas recuperadas e requisições médias por problema qualificam o processo iterativo do CEGIS Semântico.

	Baseline	CEGIS Semântico				CEGIS + Anti-Trapaça	
Modelo	Acurácia (%)	Acurácia (%)	Δ (pp)	Recuperadas	Req./Tarefa	Acurácia (%)	Δ (pp)
`Grok 4.6`	88	92	+4	4	1.13	92	+0
`DeepSeek V4 Flash 0731`	74	86	+12	12	1.25	86	+0
`GPT-5.6 Luna`	58	77	+19	19	2.00	79	+2
`Claude Sonnet 5`	32	62	+30	30	2.90	60	-2
`Gemini 3.5 Flash Lite`	21	43	+22	22	3.23	45	+2
`Gemini 3.1 Flash Lite`	12	31	+19	19	4.07	24	-7
`Leanstral 1.5.1`	3	7	+4	4	4.69	9	+2
`GPT-5.4 Nano`	1	5	+4	4	4.89	3	-2

Tabela 2: Testes de significância estatística pareada (permutação e Bootstrap, $B = 10.000$) e decomposição de erros sob feedback semântico ($\alpha = 0{,}05$).

Modelo	Δ_{CEGIS}	p_{perm} (CEGIS)	p_{boot} (CEGIS)	IC 95% Bootstrap	Sobreajuste Espúrio	Teto Repres.
`Grok 4.6`	+4.0%	0.0603	0.0431	[1.0%, 8.0%]	8	0
`DeepSeek V4 Flash 0731`	+12.0%	0.0005	0.0007	[6.0%, 19.0%]	10	4
`GPT-5.6 Luna`	+19.0%	0.0001	0.0001	[12.0%, 27.0%]	10	13
`Claude Sonnet 5`	+30.0%	0.0001	0.0001	[21.0%, 39.0%]	7	31
`Gemini 3.5 Flash Lite`	+22.0%	0.0001	0.0001	[14.0%, 30.0%]	10	47
`Gemini 3.1 Flash Lite`	+19.0%	0.0001	0.0001	[12.0%, 27.0%]	7	62
`Leanstral 1.5.1`	+4.0%	0.0603	0.0457	[1.0%, 8.0%]	0	93
`GPT-5.4 Nano`	+4.0%	0.0603	0.0496	[1.0%, 8.0%]	0	95

amostragem compreendeu 100 tarefas de treino do ARC-AGI-1 selecionadas pela ordenação lexicográfica dos identificadores hexadecimais (`00576224` a `3aa6fb7a`) gerados na criação do corpus (Chollet 2019). A inferência empregou temperatura $T = 0{,}2$, `top_p` $= 1{,}0$, limite máximo de 4.096 tokens de geração e sementes determinísticas nos provedores suportados.

5 Resultados Experimentais e Análise

O protocolo produziu 800 execuções pareadas (100 tarefas × 8 modelos). A Tabela 1 sintetiza a acurácia global e o custo operacional por modelo, enquanto que a Tabela 2 detalha os testes inferenciais e os modos de falha.

5.1 Eficácia do CEGIS sobre a Passada Única

A Tabela 2 confirma que o CEGIS supera significativamente o baseline de passada única (*pass@1*) em todos os oito modelos avaliados ($+14{,}3$ pp no agregado, $p_{\text{boot}} < 0{,}05$, com intervalos bootstrap estritamente positivos). Sob a política de parada estrita, soluções bem-sucedidas no treino no Turno 0 são preservadas sem alteração, recuperando 114 tarefas previamente não resolvidas e validando a eficácia empírica do refinamento iterativo.

5.2 Decomposição dos Modos de Falha

A classificação das 397 falhas revela que, embora o teto de busca e representação responda por $86{,}9\%$ dos erros no agregado (345 casos), o sobreajuste espúrio constitui a barreira primária nos modelos de fronteira, respondendo por 100% das falhas no `Grok 4.6` (8 erros) e $71{,}4\%$ no `DeepSeek V4` (10 erros). Em contrapartida, redes compactas falham exclusivamente pela não convergência no treino (95 erros no `GPT-5.4 Nano` e 93 no `Leanstral 1.5.1`).

A auditoria estrutural do código revelou ainda uma disparidade sistemática no comprimento dos programas segundo o desfecho: funções associadas a sobreajuste espúrio apresentaram em média $55{,}9$ linhas de código (SLOC), contra $34{,}6$ linhas nas soluções corretas de primeira tentativa. Esse aumento reflete o acúmulo de remendos heurísticos e ramificações condicionais adicionais ao longo das iterações.

5.3 CEGIS vs. Anti-Trapaça: Teste da Hipótese Central

A Tabela 3 confronta diretamente o CEGIS padrão com a variante Anti-Trapaça. No agregado, a acurácia foi de $50{,}4\%$ contra $49{,}8\%$ (diferença de $-0{,}6$ pp). A matriz de contingência pareada contabilizou 45 casos discordantes: $b = 25$ tarefas resolvidas exclusivamente pelo CEGIS e $c = 20$ resolvidas com exclusividade pelo Anti-Trapaça. O teste binomial exato de McNemar resultou em $p = 0{,}5515$, enquanto que o intervalo bootstrap contém o zero ($IC_{95\%}[-2{,}25\%, +1{,}00\%]$).

Após o controle de Bonferroni para múltiplas comparações ($\alpha/8 = 0{,}00625$), nenhum modelo isolado atingiu significância (o menor valor, registrado no `Gemini 3.1 Flash Lite` com $p_{\text{exact}} = 0{,}0156$, ajusta-se para $p_{\text{bonf}} = 0{,}1250$). Portanto, refuta-se a hipótese de que instruções textuais anti-hardcoding ofereçam vantagem generalizável. A Figura 1

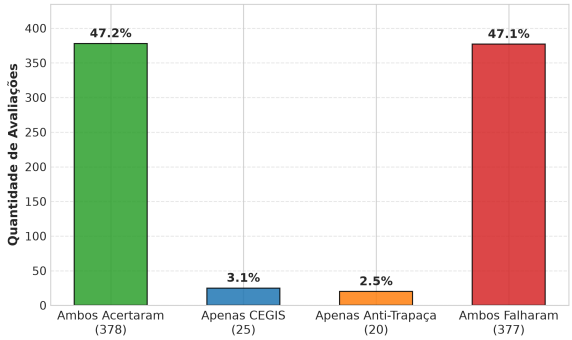


Figura 1: Distribuição das execuçõespareados entre CEGIS e Anti-Trapaça evidenciando a paridade de desempenho entre os modelos.

resume essa equivalência estrutural.

6 Discussão

A avaliação experimental conduz a três implicações teóricas e empíricas:

1. Paridade do Anti-Trapaça e Ausência de Patologia no Baseline: A equivalência estatística observada entre o CEGIS padrão e a variante Anti-Trapaça decorre de um diagnóstico empírico revelador: os modelos raramente fraudam as demonstrações de treino por meio de coordenadas cartesianas fixas. A auditoria realizada por expressões regulares sobre as 2.398 funções geradas identificou apenas 7 ocorrências ($0{,}29\%$) de coordenadas ad-hoc. Contudo, essa taxa afere estritamente a indexação literal de coordenadas (e.g., `if r == 3 and c == 5`). Contrastando esse baixo percentual com o inchaço estrutural identificado na Seção 5 ($55{,}9$ vs. $34{,}6$ SLOC), constata-se que os modelos sobreajustam por meio de remendos heurísticos distribuídos, laços e condicionais fragmentadas que acomodam os pares de treino sem capturar o invariante global, e não por checagens pontuais de coordenadas. Assim, a restrição a coordenadas literais atacou uma patologia marginal, deixando o sobreajuste heurístico desregulado. Além disso, sem barreiras sintáticas formais no sandbox, diretivas puramente textuais permanecem vulneráveis à negligência instrucional sob raciocínio complexo.

2. Esgotamento do Orçamento de Busca vs. Teto de Representação: Embora o teto de busca e representação responda por $86{,}9\%$ dos erros no agregado, o sobreajuste espúrio constitui o obstáculo preponderante na fronteira (100% no `Grok 4.6` e $71{,}4\%$ no `DeepSeek V4`). Para os modelos limitados pelo teto de busca, a não convergência no treino não reflete necessariamente incapacidade representacional inerente. O protocolo operou sob amostragem quase determinística ($T = 0{,}2$), busca estritamente linear (sem retrocesso) e orçamento restrito de $K_{\max} = 5$ turnos, aprisionando o sintetizador em mínimos locais após o primeiro contraexemplo. Essa limitação aponta que as falhas podem ser causadas pela estratégia de exploração e orçamento de busca, superáveis por busca em árvore (*Tree Search*) ou amostragem paralela, e não unicamente da capacidade intrínseca das redes.

3. Ameaça à Validade Externa e Contaminação Prévia: Uma salvaguarda metodológica crítica diz respeito à validade externa dos resultados. Como o corpus público do ARC-AGI-1 foi disponibilizado em 2019, é plausível que modelos de fronteira tenham sido expostos ao conjunto de treino e a repositórios comunitários durante o pré-treinamento. Embora o protocolo de CEGIS avalie a capacidade de induzir código sob contraexemplos em tempo de execução, não se pode descartar que a alta acurácia inicial reflita memorização prévia de primitivas visuais.

7 Limitações e Trabalhos Futuros

As descobertas reportadas delineiam quatro direções prioritárias: (i) **Escala de Amostragem:** estender o estudo às 800 tarefas completas do ARC-AGI-1 e ao ARC-AGI-2; (ii) **Imposição:** substituir diretivas puramente textuais por filtros estáticos rígidos via AST que barrem ramificações ad-hoc antes do sandbox; (iii) **Busca em Árvore:** transcender a busca linear básica mediante *Monte Carlo Tree Search* (MCTS) ou amostragem *Best-of-*N com escalonamento térmico ($\Delta T > 0$); e (iv) **DSLs Geométricas:** atenuar o teto de representação suprindo primitivas de simetria e conectividade morfológica em vez de Python puro.

8 Conclusão

Este trabalho estabeleceu uma investigação estatística acerca da síntese indutiva de programas guiada por contraexemplos no ARC-AGI-1, confrontando o protocolo clássico de auto-reparo conversacional com diretivas textuais restritivas anti-hardcoding. A evidência empírica coletada sobre oito modelos de linguagem refutou a hipótese de que as diretivas de prompt negativas utilizadas promovam maior generalização para grades ocultas, demonstrando equivalência estatística entre as abordagens.

A análise dos programas sintetizados revelou que essa paridade decorre do viés algorítmico intrínseco dos grandes modelos de linguagem, que raramente recorrem a ramificações ad-hoc por coordenadas no baseline ($0{,}29\%$), tornando a restrição textual pontual ineficaz frente ao verdadeiro mecanismo de sobreajuste espúrio: a inflação estrutural de código por acúmulo de remendos.

Por fim, a taxonomia de erros evidenciou que a quase totalidade das falhas origina-se da não convergência sobre as demonstrações de treino, e não de sobreajustes espúrios. Esse achado demonstra que a fronteira de avanço no ARC-AGI não depende primariamente de ajustes sutis de prompt ou penalizações de heurísticas, mas da ampliação do orçamento computacional em tempo de inferência via busca estruturada em árvore e do suporte a linguagens específicas de domínio.

Tabela 3: Testes de significância pareada para Anti-Trapaça vs. CEGIS (Binomial Exato, Permutação e Bootstrap $B = 10{,}000$, $\alpha = 0{,}05$). Coluna b/c: pares discordantes (b: exclusivas do CEGIS; c: exclusivas do Anti-Trapaça).

Modelo	CEGIS	Anti-Trapaça	Δ	b/c	p_{exact}	p_{perm}	IC 95% Bootstrap	p_{bonf}
`Claude Sonnet 5`	62.0%	60.0%	-2.0%	5 / 3	0.7266	0.7236	[-8.0%, +3.0%]	1.0000
`DeepSeek V4 Flash 0731`	86.0%	86.0%	+0.0%	0 / 0	1.0000	1.0000	[+0.0%, +0.0%]	1.0000
`GPT-5.4 Nano`	5.0%	3.0%	-2.0%	3 / 1	0.6250	0.6243	[-6.0%, +2.0%]	1.0000
`GPT-5.6 Luna`	77.0%	79.0%	+2.0%	2 / 4	0.6875	0.6875	[-2.0%, +7.0%]	1.0000
`Gemini 3.1 Flash Lite`	31.0%	24.0%	-7.0%	7 / 0	0.0156	0.0178	[-12.0%, -3.0%]	0.1250
`Gemini 3.5 Flash Lite`	43.0%	45.0%	+2.0%	6 / 8	0.7905	0.7919	[-5.0%, +9.0%]	1.0000
`Grok 4.6`	92.0%	92.0%	+0.0%	0 / 0	1.0000	1.0000	[+0.0%, +0.0%]	1.0000
`Leanstral 1.5.1`	7.0%	9.0%	+2.0%	2 / 4	0.6875	0.6861	[-2.0%, +7.0%]	1.0000
TOTAL AGREGADO	50.4%	49.8%	-0.6%	25 / 20	0.5515	0.5501	[-2.2%, +1.0%]	—

Referências

Acquaviva, S.; Pu, Y.; Kryven, M.; Sechopoulos, T.; Wong, C.; Ecanow, G.; Nye, M.; Tessler, M. H.; and Tenenbaum, J. B. 2022. Communicating Natural Programs to Humans and Machines. In *Advances in Neural Information Processing Systems*, volume 35, 3731–3743.

Chollet, F. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547*.

Ellis, K.; Wong, L.; Nye, M.; Sablé-Meyer, M.; Cary, L.; Pozo, L. A.; Hewitt, L.; Solar-Lezama, A.; and Tenenbaum, J. B. 2023. DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake–Sleep Bayesian Program Learning. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 381(2251): 20220050.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 46534–46594. Curran Associates, Inc.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: language agents with verbal reinforcement learning. In Oh, A.; Naumann, T.; Globerson, A.; Saenko, K.; Hardt, M.; and Levine, S., eds., *Advances in Neural Information Processing Systems*, volume 36, 8634–8652. Curran Associates, Inc.

Solar-Lezama, A. 2008. *Program Synthesis by Sketching*. Ph.D. thesis, University of California, Berkeley.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; ichter, b.; Xia, F.; Chi, E.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In Koyejo, S.; Mohamed, S.; Agarwal, A.; Belgrave, D.; Cho, K.; and Oh, A., eds., *Advances in Neural Information Processing Systems*, volume 35, 24824–24837. Curran Associates, Inc.